

الجامعة الإسلامية

بغزة

كلية تكنولوجيا

المعلومات



The Islamic University of Gaza

Faculty of IT

LinkHub

**A desktop application to control all your devices
and files from one place**

**تطبيق سطح مكتب للتحكم في جميع
اجهزتك من مكان واحد**

By

AbdulKhalik AbdulKarim Habib – 120221309

Abdullah Ibrahim Nassar - 120201447

Mohammed Mazen Alghazali – 120200625

Dawood Mohammed Dawood – 120191687

Suhaib Eyad Al-Basyouni - 120203757

Dr. Iyad Al-Agha

**A graduation project report submitted in partial
fulfillment of the requirements for the degree of
Bachelor of Information Technology**

Feb/2026

Abstract

In today's multi-device landscape, users struggle with fragmented workflows due to either restrictive proprietary ecosystems or functionally siloed cross-platform tools. To address this, we present **LinkHub**, a unified desktop application that serves as a central command center for device control, screen mirroring, and media management. Unlike existing solutions, LinkHub integrates a proactive media engine that enables users to retrieve web content and route it directly to connected Android devices in a single, streamlined action. The system consolidates device pairing, screen casting, file sharing, and web media retrieval into one intuitive dashboard. This project demonstrates that a modular, open approach can effectively break hardware monopolies, providing a privacy-focused, vendor-agnostic alternative that significantly reduces user effort and minimizes task-switching overhead.

ملخص الدراسة

في بيئة اليوم متعددة الأجهزة، يعاني المستخدمون من تشتت سير العمل بسبب الأنظمة الاحتكارية المقيدة أو الأدوات متعددة المنصات المنعزلة وظيفياً. لمعالجة هذه الفجوة، نقدم **LinkHub**، تطبيق سطح مكتب موحد يعمل كمركز تحكم رئيسي لإدارة الأجهزة، وعكس الشاشة، وتوجيه الوسائط. على عكس الحلول التقليدية، يدمج التطبيق محركاً استباقياً لاسترجاع محتوى الويب وتوجيهه مباشرة إلى أجهزة أندرويد المتصلة في خطوة واحدة مبسطة. يجمع النظام مهام إقران الأجهزة، وبث الشاشة، ومشاركة الملفات، واسترجاع وسائط الويب في لوحة تحكم بديهية واحدة. يُظهر هذا المشروع أن النهج المفتوح والمعياري قادر على كسر احتكار الأجهزة، مقدماً بديلاً احترافياً يركز على الخصوصية ولا يرتبط بمورد محدد، مما يقلل بشكل كبير من جهد المستخدم ويخفض تكلفة تبديل المهام.

Dedication

إلى رموز الرجولة والتضحية إلى من

دفعونا إلى العلم و بهم ازداد افتخارنا

آباؤنا

إلى من تتسابق الكلمات لتخرج معبرة عن مكنون ذاتها إلى

من تسعد قلوبنا بلقياهم إلى روضات الحب التي تنبت أزهى الأزهار

أمهاتنا

إلى هذا الصرح العلمي الفتي والجبار

الجامعة الإسلامية بغزة

Acknowledgment

Praise be to God, praise be to God for our ability to complete this work.

We would like to express our gratitude to our supervisor, Dr. Iyad Al-Agha, who did his utmost to encourage us to deliver excellent work and provided our group with valuable information and advice to improve each time. We thank our families and parents who supported us and provided us with comfort and support during the past four years. Finally, we would like to thank everyone who helped, supported, and encouraged us to
.successfully complete our graduation project

Table of Contents

Right-click here and choose "Update Field" (or press Ctrl+A then F9) to generate the Table of Contents.

List of Tables

3	Table (1): Number of Laptop vs. Mobile Users (2021-2023)
3	Table (2): Test cases runner customization parameters.

List of Figures

- Figure (1): Average time on site (mobile vs desktop)
- Figure (6.1): Layered Architecture Diagram
- Figure (6.2): Main Interface
- Figure (6.3): Format Selection Dialog
- Figure (6.4): Download Queue
- Figure (6.5): Downloads Tab
- Figure (6.6): Device Details Window
- Figure (6.7): Screen Mirroring
- Figure (6.8): Device Selection Modal

List of Abbreviations

Android Debug Bridge	ADB
Application Programming Interface	API
Cascading Style Sheets	CSS
Class Under Test	CUT
Graphical User Interface	GUI
HyperText Markup Language	HTML
Institute of Electrical and Electronics Engineers	IEEE
JavaScript	JS
Local Area Network	LAN
Operating System	OS
Request for Comments	RFC
Universal Serial Bus	USB
Wide Area Network	WAN

Chapter 1

Introduction

In today's digital landscape, smart devices such as smartphones, laptops, and tablets have become indispensable tools. This pervasive adoption has given rise to the era of "multi-device ownership," with recent studies indicating that over 65% of internet users regularly operate at least two or three devices [1]. This trend is especially pronounced in professional and technical sectors, where more than 90% of managers and technicians rely on multiple devices to manage their daily workflows [2]

However, achieving seamless integration across this heterogeneous hardware landscape remains a persistent challenge. Current solutions predominantly fall into two categories. The first comprises proprietary hardware ecosystems, such as Apple's Continuity and Samsung's Galaxy ecosystem. While these systems deliver exceptional integration, they fundamentally **monopolize the seamless user experience**, creating a "vendor lock-in" that forces users to remain within a single brand's walled garden to access premium connectivity features [3][4]. The second category consists of cross-platform software solutions like LocalSend and Intel Unison. Although they offer brand-agnostic connectivity, they are **functionally siloed**; each tool provides only one or two isolated utilities—such as standalone file transfer or screen mirroring—while critically lacking an integrated mechanism for retrieving and routing web content directly to devices [5][6]

This fragmentation reveals a clear gap: the absence of a unified platform that consolidates device control, screen mirroring, and web content management.

LinkHub bridges this gap by serving as a **centralized command center** for all connected devices. Unlike conventional tools, LinkHub introduces an "**Active Media Node**" engine, which uniquely automates the entire pipeline—from downloading media directly from the internet to routing it instantly to any connected Android device—in a single click. By leveraging **ADB over USB** for stable, low-latency wired communication and Electron.js for a cross-platform interface, LinkHub eliminates the need for separate download managers and manual file transfers. Ultimately, the application consolidates fragmented workflows into a single, intuitive dashboard, significantly reducing user effort and maximizing productivity

1.1 Problem Statement

The proliferation of interconnected devices has introduced significant friction in maintaining a smooth digital workflow [7]. Despite the availability of numerous connectivity tools, users continue to face four persistent, interrelated challenges

Functional Fragmentation: Existing software solutions are siloed—offering only isolated utilities such as file transfer or screen mirroring—forcing users to juggle multiple independent applications to complete a single cross-device task

Ecosystem Lock-in: Seamless, high-quality integration is exclusively available within proprietary ecosystems (e.g., Apple and Samsung). This "walled garden" approach creates a vendor lock-in, penalizing users who own devices from different manufacturers with poor or non-existent interoperability

Platform Heterogeneity: The diversity of operating systems (Windows, Linux, Android) introduces inherent synchronization gaps, making real-time data transfer and device control complex, unreliable, and inefficient compared to homogenous environments

Web-to-Device Disconnect: There is a notable absence of a system that treats the internet as an integrated node within the local network. Currently, retrieving media from the web and transferring it to a target device is a fragmented, multi-step process requiring separate download managers and manual file transfers

Collectively, these issues create a fragmented user experience where productivity is hindered by context switching and redundant manual operations

1.2 Objectives

To address the identified challenges of functional fragmentation, ecosystem lock-in, platform heterogeneity, and the web-to-device disconnect, this project establishes one main objective supported by five specific technical sub-objectives

1.2.1 Main Objective

To develop **LinkHub** as a centralized command center that unifies device connectivity, screen mirroring, and web-media routing within a single cross-platform desktop application, delivering efficient and automated workflows on both Windows and Linux environments

1.2.2 Sub-Objectives

To achieve the main objective, the following technical sub-objectives are defined

Unified User Interface: Design and implement a modern **.1**

graphical user interface using Electron.js that consolidates all tools—device management, screen mirroring, and media downloading—into a single central

dashboard, eliminating the need to switch between multiple standalone applications (addressing functional fragmentation)

Stable Wired Connectivity: Establish a robust, low-latency communication layer using **ADB over USB** to ensure stable screen mirroring, responsive device control, and high-speed file transfers, prioritizing connection reliability over wireless flexibility (addressing platform heterogeneity and instability) **.2**

Active Media Node Engine: Develop a seamless one-click download and routing system that treats web platforms as an active network node. This engine automatically routes downloaded internet media to any connected Android device, eliminating the multi-step process of separate download managers and manual file transfers (addressing the web-to-device disconnect) **.3**

Persistent Data Management: Integrate a local SQLite database to persistently manage task logs, device configurations, and download histories, ensuring data continuity, organized content tracking, and session recovery across application restarts **.4**

Cross-Platform Compatibility: Package and validate the application for both Windows and Linux operating systems, ensuring consistent functionality and user experience regardless of the desktop environment, thereby providing a vendor-agnostic alternative to proprietary ecosystems (addressing ecosystem lock-in) **.5**

1.3 Scope and Limitations

1.3.1 Project Scope

The scope of this project defines the core features, target platforms, and supported devices that are explicitly included within the current implementation of LinkHub

- **Platform Support:** A cross-platform desktop application fully compatible with **Windows** and **Linux** operating systems
- **Mobile Device Compatibility:** Supports **Android devices** (version 11.0 and above). iOS devices are explicitly excluded
- **Language Support:** The user interface supports the **Arabic language** as the primary interface language
- **Connectivity Protocol:** Establishes a **secure, high-speed wired connection** between the PC and Android devices using **ADB over USB**, prioritizing connection stability and low-latency performance
- **Screen Mirroring & Control:** Provides real-time screen casting and **direct device control** (mouse and keyboard input) via the USB connection
- **Active Media Node:** Enables **one-click downloading** of media and files directly from the internet and instant routing to any connected Android device
- **Local Data Persistence:** Utilizes an **SQLite database** to persistently store task logs, download histories, and device configurations

1.3.2 Project Limitations

The following constraints define the boundaries and known restrictions of the current system version

iOS Incompatibility: The system does not support iOS devices •
due to Apple's closed operating system architecture, which restricts low-level debugging, ADB-equivalent protocols, and screen mirroring capabilities [8]

Wired-Only Connectivity: This version operates **exclusively** •
via USB cable. Wireless connectivity (Wi-Fi/ADB over TCP/IP) was intentionally excluded to prioritize connection stability, lower latency, and development feasibility within the given project timeline

Android Version Dependency: The system requires Android •
11.0 or higher to ensure consistent ADB functionality and screen mirroring performance

External Tool Dependencies: Core functionalities rely on •
third-party open-source tools (**yt-dlp** for media downloading and **screpy** for screen mirroring). Regular maintenance and updates are required to ensure ongoing compatibility with changes in web platform structures, Android OS updates, and tool API modifications

1.4 Importance of the Project

LinkHub addresses the critical gaps identified in the Problem Statement—functional fragmentation, ecosystem lock-in, platform heterogeneity, and the web-to-device disconnect—by delivering a unified desktop solution that consolidates device control, screen mirroring, and media routing into a single workflow. Its significance is demonstrated through four key contributions

Unified Device Management: LinkHub functions as a centralized command center, integrating file transfer, screen mirroring, and device control within a single interface. This eliminates the cognitive overhead and time lost switching between multiple standalone tools, directly resolving the issue of functional fragmentation

Active Media Node Innovation: Unlike conventional tools limited to passive file transfers, LinkHub introduces a proactive "Active Media Node" engine. This unique feature enables one-click downloading of internet media and instant routing to any connected Android device—collapsing the traditional multi-step process (download

→ locate → transfer) into a single action, thereby solving the web-to-device .disconnect

Vendor-Agnostic Cross-Platform Compatibility: By delivering consistent, high-performance functionality on both Windows and Linux via Electron.js, LinkHub provides a practical alternative to proprietary ecosystems. It breaks the "walled garden" monopoly by offering seamless integration without forcing users into a single .hardware brand, directly countering ecosystem lock-in

1.5 Tools and Equipment's

The development of LinkHub leveraged a modular toolchain spanning frontend, backend, database, and testing environments. The following tools and technologies :were utilized

:Development Environment & Runtime

Node.js v20.x: Chosen for its non-blocking, event-driven •
architecture, enabling efficient management of multiple backend processes
(ADB communication, media downloading, file routing) within a single
.runtime environment [11][12]

Electron.js Framework: Used to package the web-based •
frontend into a native cross-platform desktop application, enabling code reuse
across Windows and Linux while maintaining separation between UI and
.backend logic [14]

:Frontend Technologies

HTML5, CSS3, JavaScript (ES6), Bootstrap 5: Standard •
web technologies for building a responsive, modern user interface, with
.Bootstrap accelerating layout and component design

:Backend & Persistence

SQLite3 (via better-sqlite3): Embedded relational database •
requiring no server configuration, used for persistently storing task logs,
download histories, device settings, and session data across application restarts
.[13]

:Third-Party Integration Tools

yt-dlp: A powerful open-source command-line tool for •
downloading media (video/audio) from over 1,000 web platforms. Integrated
.as the "Active Media Node" engine to fetch internet content [15]

scrcpy: An open-source tool leveraging the ADB protocol to •
enable real-time screen mirroring and touch/mouse control of Android devices
from a desktop. Operates exclusively over USB for optimal latency and
.stability [16]

:Testing & Virtualization

VMware Workstation Player: Used to create isolated virtual •
machines for Windows and Linux, enabling systematic cross-platform testing
.without requiring dual-boot or physical hardware changes [18]

1.6 Methodology:

For this project, we will use the **Agile (Scrum)** methodology to manage the development process. This approach allows us to break the project down into manageable, iterative development cycles (Sprints). We chose this methodology for its flexibility, enabling us to continuously develop, test, and refine features while catching bugs early and ensuring steady progress.

The process follows these steps:

- The development process follows these core steps: Requirements Analysis & Domain Modeling: Defining the core capabilities of LinkHub (such as device management, media downloading via yt-dlp, file transfer via ADB , and screen mirroring via scrcpy) using Domain-Driven Design (DDD) principles. .
- Architecture & Layered Development: Building the application based on a 5-Layer Clean Architecture (Presentation, Application, Domain, Infrastructure, and Runtime). The desktop GUI is built using Electron.js , backed by Node.js and an SQLite database (better-sqlite3) for persistent data.
- **Testing & Validation:** Writing unit and end-to-end (E2E) tests using **Jest** and mock file systems (**memfs**) to ensure cross-platform compatibility and system stability.
- Review and synchronization: Regularly review progress and test features and functions to ensure project deliverables are met on schedule.

1.7 Timeline

A Sprint-based cycle system will be followed for project development to achieve maximum flexibility and maintain software quality:

Month One: Core Architecture, Device Management & Security Domain

Sprint 1: "System Architecture & USB/ADB Device Integration"

- Analysis and Design: Defining the Electron.js interface requirements and drawing a data flow diagram for pairing via ADB[22].
- Development: Setting up a Docker environment and building the basic interface with ADB over TCP/IP enabled[23].
- Result: A desktop application that can wirelessly recognize an Android phone after the initial pairing.

The second course (Sprint 2): "Security Layer"

- Configuring and securing the connection: Designing an SSH tunnel architecture to ensure the encryption of commands and files[24].
- Development: Integrating the SSH tunnel into Node.js and developing a session management system in an SQLite database.
- Result: A fully encrypted connection between the phone and the computer to protect user privacy.

Month Two: Control and Media Features

Third session (Sprint 3): "Screen mirroring and wireless control"

- Analysis and Design: Performance requirements were analyzed to minimize screen latency.
- Development: The Scrcpy tool was integrated, and a control interface was developed to receive mouse and keyboard input.
- Result: The phone screen was displayed on the computer and could be controlled wirelessly with a single click.

Course 4 (Sprint 4): "Active Media Node"

- Analysis and Design: Designing a workflow for retrieving and processing links from the browser across the system.
- Development: Integrating the yt-dlp tool and programming the download engine and automatic file routing.
- Result: A feature that allows downloading videos from a web link and instantly sending them to any connected device.

Third month: Integration, quality, and delivery

The fifth course (Sprint 5): "Task Management and Automation"

- Analysis and Design: Design of the Task Logs interface and a custom shortcut system.
- Development: Implementation of the control panel for device management (add/delete) and programming of keyboard shortcuts.
- Result: An integrated system for managing past tasks and speeding up work through shortcuts.

Sprint 6: "Platform Testing and Final Documentation"

- Analysis and Design: Analyze performance results on Windows and Linux and identify any remaining gaps.

- Development: Fix bugs resulting from VMware tests and use Wireshark to ensure data integrity.
- Outcome: A stable final product with a complete project report and IEEE-formatted references.

Outputs of each cycle (Sprint Artifacts):

- Sprint Backlog: A list of analytical and programming tasks for each cycle.
- Working Increment: A version of the application that includes the new feature in addition to the above.
- Sprint Review: A live demonstration of the feature in front of the supervisor (Dr. Iyad Al-Agha).

Note: The user interface and user experience will be redesigned and improved in each course.

Chapter 2

Related Works

2.1 State of the Art

In the modern digital era, the concept of "Device Interoperability" has evolved into two distinct technological trajectories: Proprietary Ecosystems and Cross-Platform Software Solutions [25]. Understanding these trajectories is essential to identifying the current benchmarks in seamless device integration

Proprietary Ecosystems: Industry leaders like Apple and Samsung have established new standards for integrated experiences. These systems leverage deep hardware-software co-design to deliver features such as unified clipboards, instant file synchronization, and peripheral sharing (e.g., using a single mouse across multiple devices). However, this integration comes at a cost: these systems are technically "closed," creating a "walled garden" where access to premium features is restricted exclusively to devices within the same brand, effectively imposing a vendor lock-in on users [26]

Cross-Platform Software Solutions: In parallel, a growing trend toward "brand-agnostic" solutions has emerged. These projects aim to deliver ecosystem-like experiences regardless of hardware choices, utilizing open protocols (such as ADB) and modern web frameworks (e.g., Electron.js)

to build unified interfaces that consolidate various functions—file transfer, screen mirroring, and media retrieval—into a single dashboard

While proprietary systems offer stability, they enforce vendor dependency. Conversely, cross-platform tools offer flexibility but often remain functionally fragmented, focusing on one or two isolated utilities. Both trajectories critically lack an integrated mechanism for directly retrieving and routing web content to connected devices—a gap this project explicitly addresses

2.2 Review of Related Software Solutions

This section reviews prominent cross-platform applications that provide device connectivity and file-sharing capabilities, identifying their strengths and the functional gaps that distinguish LinkHub

1. Intel Unison

Intel Unison is a proprietary application developed to synchronize mobile devices (Android and iOS) with Intel-based personal computers [27]

Key Features: Offers an elegant graphical interface for

- sending/receiving text messages, managing mobile notifications, and wireless file transfer

Limitations: Highly optimized for Intel-certified hardware, causing performance degradation on other processors. Critically, it lacks screen mirroring capabilities, provides no native Linux support, and offers no integrated web content retrieval mechanism [28]

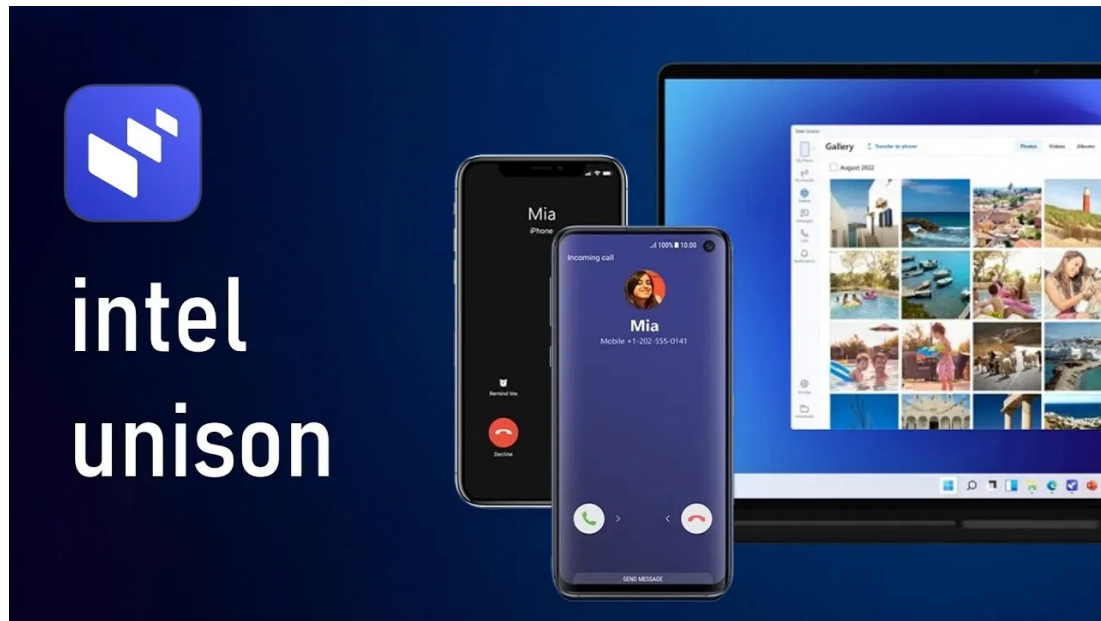


Figure (2.1): Intel Unison

2. LocalSend

LocalSend is a widely adopted open-source application focused on secure, privacy-first file sharing within local networks [29]

Key Features: Simple, intuitive GUI with seamless cross-platform support (Windows, Linux, Android, iOS). Requires no internet or .cloud accounts

Limitations: Exclusively a file-transfer tool. It does not offer screen mirroring, device control, or any web-content downloading or routing .functionality

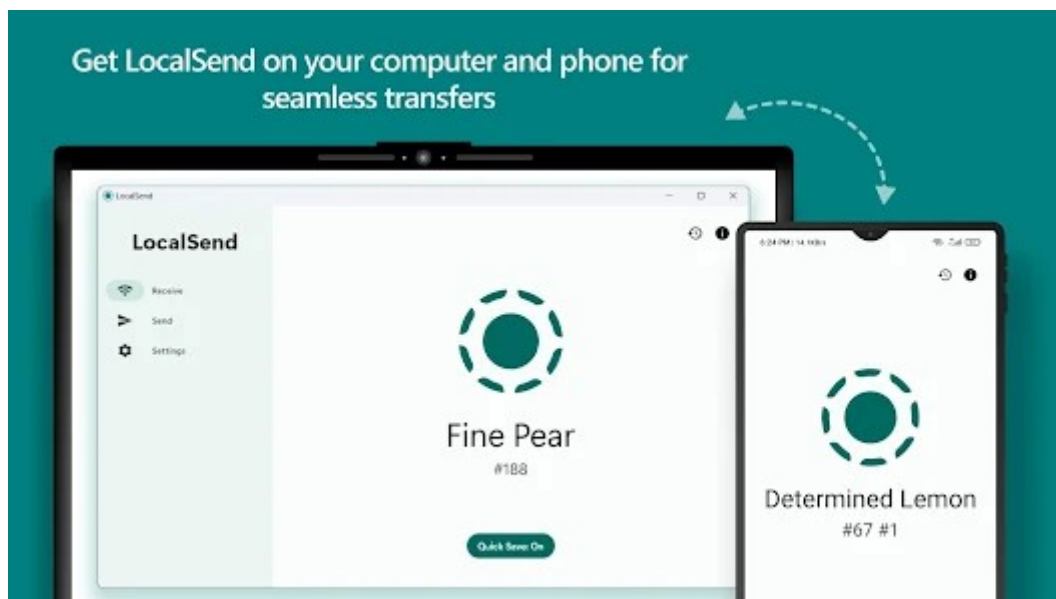


Figure (2.2): LocalSend

3. KDE Connect

KDE Connect is a well-established open-source project for linking Android devices to desktops, particularly popular in the Linux community (though a Windows version exists) [30]

Key Features: Offers a broad range of features including shared clipboard, remote input (using the phone as a mouse), and notification synchronization

Limitations: Configuration can be complex for non-technical users. It lacks a built-in high-performance screen mirroring engine, does not provide automated web-content routing, and its interface is fragmented across multiple settings panels

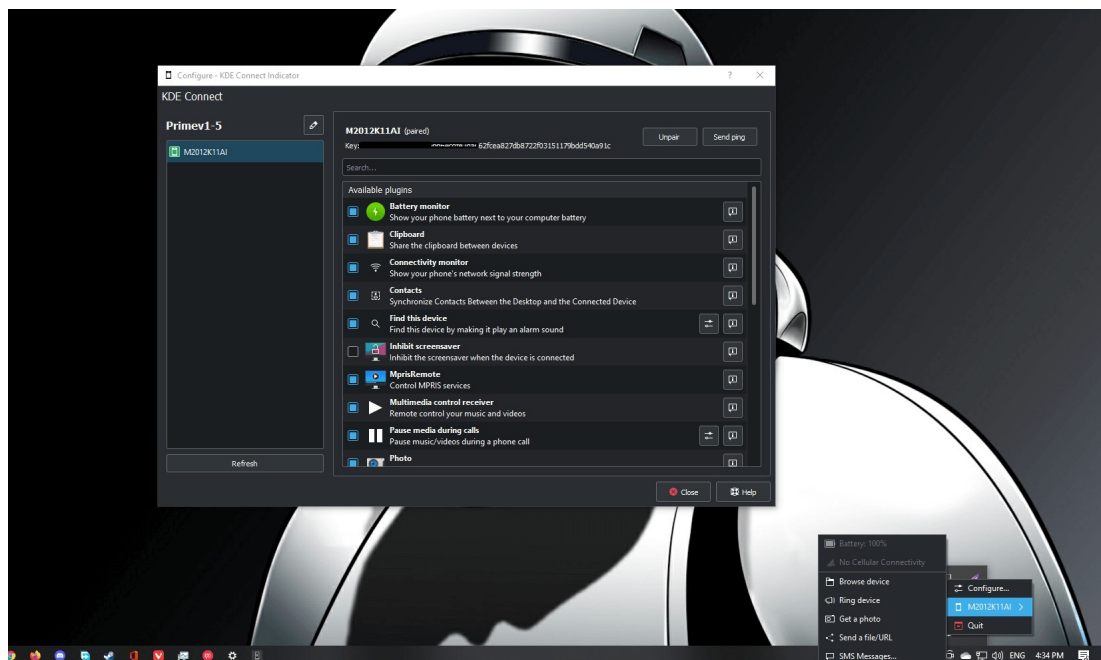


Figure (2.3): KDE Connect

Comparative Analysis

Table (1) summarizes the key functional differences between the reviewed tools and :LinkHub

LinkHub	KDE Connect	LocalSend	Intel Unison	Feature
Windows, Linux, Android	Windows, Linux, Android	Windows, Linux, Android, iOS	Windows, Android, iOS	Platform Support
USB Cable (ADB)	Wi-Fi (Same Network)	Wi-Fi (Same Network)	Wi-Fi / Bluetooth	Connection Technology
Supported (via scrcpy)	Limited (Mouse only)	Not Supported	Not Supported	Screen Mirroring
Full Control (Mouse/Keyboard)	Limited (Remote Input)	Not Supported	Not Supported	Device Control
Supported (Download + Route)	Not Supported	Not Supported	Not Supported	Active Media Node
Physical (USB) / N/A	TLS	TLS	TLS	Data Encryption
Supported (Shortcuts, Logs)	Limited	Not Supported	Not Supported	Task Automation
None (Offline/Local)	None (Local Only)	None (Local Only)	Requires Internet (sometimes)	Internet Dependency
Supported	Supported	Supported	Not Supported	Linux Support

2.3 Summary and Gap Analysis

The reviewed tools demonstrate the current landscape of cross-platform device connectivity

Intel Unison offers polished UX but is hardware-restricted and lacks mirroring •

LocalSend excels at file sharing but is functionally siloed •

KDE Connect is feature-rich but fragmented and lacks dedicated mirroring or web routing •

Collectively, these tools suffer from three critical gaps

Functional Fragmentation: No single tool offers combined device control, screen mirroring, and media routing .1

Web-to-Device Disconnect: None of the reviewed tools provide an integrated mechanism to download content directly from the web and route it to devices within the same workflow .2

Platform Limitations: Intel Unison lacks Linux support; KDE Connect's configuration is complex; LocalSend is single-purpose .3

LinkHub bridges these gaps by

Delivering a **unified dashboard** that consolidates device management, screen mirroring, and media routing •

Introducing the **Active Media Node** engine, enabling one-click web media retrieval and instant device routing—a feature completely absent from all reviewed alternatives •

Prioritizing **USB connectivity** to ensure maximum stability and lowest latency, making it ideal for professional and development environments where reliability is paramount •

Chapter 3: Methodology

This chapter presents the systematic approach adopted to design, implement, and deliver LinkHub as a functional cross-platform desktop application. It details the software development methodology (Agile/Scrum) and the rationale behind its selection, outlines the key practices and phases followed throughout the project lifecycle, and describes the adaptations made to accommodate project-specific constraints—most notably the decision to prioritize wired USB connectivity over wireless alternatives to ensure stability and low-latency performance. Additionally, the chapter provides a comprehensive overview of the technology stack, the iterative sprint-based timeline, and the team management structure that facilitated collaborative development. Together, these elements establish a clear, replicable framework that guided the project from initial requirements to final delivery.

Software Development Methodology 3.1

The LinkHub project was developed using the **Agile (Scrum) methodology**. This iterative, incremental approach was selected to manage the inherent complexities of building a cross-platform desktop application that integrates multiple external tools (ADB, yt-dlp, scrcpy) within a layered architectural framework (Clean Architecture + Domain-Driven Design).

3.2 Rationale for Choosing Agile

While the project's layered architecture—spanning Presentation, Application, Domain, Infrastructure, and Runtime—might initially suggest a Waterfall approach (where each layer is fully completed before the next begins), several critical factors mandated the flexibility and iterative nature of Agile

Cross-Platform Compatibility Risks: The application must .1

function reliably on both Windows and Linux. In a Waterfall model, comprehensive testing occurs only at the end, risking the late discovery of platform-specific incompatibilities—potentially requiring extensive rework of foundational layers. Agile allowed us to validate compatibility incrementally, starting from the minimal viable implementation of each layer across both .platforms from the very first sprints

Integration with External "Black-Box" Tools: LinkHub .2

relies on third-party tools (yt-dlp for media downloading, screpy for screen mirroring, and ADB for device communication). These tools are essentially "black boxes" with their own behaviors, update cycles, and platform-specific quirks. Waterfall's "build-then-test" approach would expose integration issues too late, risking catastrophic failure. Agile enabled us to integrate and test each tool iteratively, starting with a minimal adapter, identifying compatibility issues early, and adapting our code incrementally as we learned about tool .behavior across different environments

Early Feasibility Validation: The project's success depended **.3**
on proving that the proposed features (device management, screen mirroring, media downloading, and routing) could work in practice across the target platforms. Agile allowed us to construct a **minimal vertical slice**—a thin implementation spanning all layers (from Infrastructure up to Presentation)—in the first sprint. This proved the technical viability of the core concept early, de-risking the project and allowing us to pivot or refine requirements based on real-world feedback, rather than discovering fundamental flaws after full .implementation

Incremental Layer Development and Integration: Contrary **.4**
to a strict Waterfall interpretation, we did not build each layer in isolation. Instead, we adopted a "**minimal layer, then evolve**" strategy. In Sprint 1, we built the minimal Infrastructure layer (enough to run ADB and basic device detection) and the minimal Application layer to orchestrate it, then tested their integration. In subsequent sprints, we *evolved* these layers—adding more sophisticated process management, state synchronization, and error handling—while simultaneously building out the Domain and Presentation layers on top of the stable foundations. This iterative layering allowed us to manage technical debt, refactor as needed, and ensure each layer's dependencies were .correctly satisfied before expanding functionality

Parallel Frontend and Backend Development: The strict **.5**
separation between the Electron frontend (GUI/CLI) and the Node.js backend allowed parallel development tracks. Agile's sprint structure enabled the frontend team to build UI components and IPC handlers in sync with the backend team's delivery of orchestrators and infrastructure services, ensuring that both sides could be integrated and tested continuously throughout the .project lifecycle

Risk Mitigation and Error Recovery: Given the complexity .6

of managing external process lifecycles (ADB daemons, yt-dlp processes, scrapy instances) and the potential for unexpected runtime errors (device disconnections, network timeouts, file system issues), Agile's iterative cycles provided a safety net. Each sprint included dedicated testing and bug-fixing phases, allowing us to recover from errors and refine error-handling logic incrementally—without the "all-or-nothing" risk of a final integration phase in .Waterfall

In essence, Agile provided the necessary **adaptability** to navigate the uncertainties of cross-platform development and third-party tool integration, while our layered architecture provided the **structural discipline** to maintain a clean, maintainable codebase. The combination allowed us to deliver a stable, feature-complete application within the project timeline while continuously validating progress and .mitigating risks

Key Practices and Phases 3.3

The Agile (Scrum) methodology was operationalized through a structured yet flexible set of practices, adapted to the project's layered architecture and cross-platform requirements. The development process was organized into six sprints, each spanning approximately 2–3 weeks, over a total project duration of five months with a team of five members. The following practices were consistently applied throughout the :project lifecycle

Sprint Planning: At the beginning of each sprint, the team collaboratively defined a Sprint Goal and selected a set of backlog items (user stories and technical tasks) aligned with that goal. Priorities were determined based on feature dependencies, risk factors, and the need to validate cross-platform compatibility early. Each sprint was designed to deliver a potentially shippable increment—a working version of the .application with at least one new functional capability

Incremental Layered Development: Rather than completing each architectural layer (Infrastructure, Application, Domain, Presentation) in isolation, the team adopted an

"evolving layers" strategy. In early sprints, we built the minimal viable version of each layer—just enough to enable basic functionality (e.g., device detection via ADB). In subsequent sprints, we iteratively enhanced each layer, adding robustness, error handling, state synchronization, and advanced features. This approach allowed us to validate layer integration continuously and avoid the risks of "big bang" integration at the end

Weekly Stand-up Meetings: The team conducted brief Weekly coordination sessions (via WhatsApp and Google Meet) to synchronize progress, identify blockers, and adjust task assignments as needed. These meetings ensured that all members were aligned on sprint goals and that impediments were addressed promptly

Continuous Integration and Testing: Each sprint included dedicated testing activities. Features were tested immediately after implementation on both Windows and Linux environments (using VMware virtual machines) to validate cross-platform compatibility. Unit tests were written for core business logic, and end-to-end tests were performed for critical user workflows (device connection, media download, file transfer). This early and frequent testing reduced the risk of late-stage integration failures

Sprint Retrospective: Following each review, the team conducted an internal retrospective session to evaluate performance, identify process improvements, and address any team dynamics or technical debt. Lessons learned were applied to optimize workflows in the next sprint, fostering a culture of continuous improvement

Parallel Frontend-Backend Development: Given the clear separation between the Electron frontend (GUI and CLI) and the Node.js backend, the team organized workstreams in parallel. Frontend developers (2 members) focused on UI components, IPC handlers, and state synchronization, while backend developers (2 members) concentrated on orchestrators, adapters, and infrastructure services. A dedicated member acted as project coordinator, managing integration and supervisor communication

This set of practices ensured that the project progressed steadily, with continuous validation of both technical feasibility and user-facing functionality, while maintaining flexibility to adapt to emerging challenges and supervisor feedback

Adaptations to the Standard Methodology 3.4

While the core Agile (Scrum) framework was followed consistently, two significant adaptations were made to align the methodology with project realities, technical constraints, and evolving requirements

Correction of Connectivity Assumption: During the initial planning phase, the team assumed that secure device communication would be achieved through SSH tunneling. However, upon deeper technical investigation, it was discovered that the ADB (Android Debug Bridge) protocol natively employs TLS encryption for secure data transmission over USB and TCP/IP connections. Consequently, the separate "Security Layer" sprint originally envisioned for implementing SSH was merged into the core connectivity sprint, eliminating redundant effort and allowing the team to focus on leveraging ADB's built-in security features rather than building an unnecessary parallel encryption layer

Elimination of Wireless Connectivity (Wi-Fi/ADB over TCP/IP): The original project scope included wireless device connectivity as a key feature. However, during early testing phases, the team identified critical challenges

Complexity of managing ADB over a wireless connection: A major issue was that the ADB tool exhibited unstable behavior in wireless environments, leading to unexpected failures that were difficult to test, given their reliance on actual network conditions—factors that cannot be accurately tested or predicted

Network instability: Wireless connections exhibited unpredictable latency spikes and disconnections, particularly in environments with congested or unreliable Wi-Fi networks

Priority on stability and low-latency: The core use cases—screen mirroring and real-time device control—demanded a stable, low-latency connection. USB provided superior performance and reliability compared to Wi-Fi

Given these challenges and the fixed project timeline, the team made a strategic decision to remove wireless connectivity from the current version and focus exclusively on wired USB connections. This adaptation allowed the team to

Deliver a stable, production-ready application within the allocated five-month timeframe

Allocate development effort to higher-value features such as the Active Media Node engine, task automation, and cross-platform compatibility

Lay a solid foundation for potential wireless support in future iterations, should time and resources permit

Streamlined Sprint Structure: As a result of the above adaptations, the originally planned Sprint 2 (Security Layer) was eliminated, and its minimal security-related tasks (session management, device trust, and user preferences) were redistributed across Sprint 1 and Sprint 5. This streamlined the sprint cadence from seven to six sprints, improving focus and reducing overhead without compromising system security—since ADB's native TLS encryption already provided the necessary data protection

These adaptations reflect the team's commitment to pragmatic decision-making and risk-aware planning, ensuring that the final deliverable met the highest priority requirements—stability, performance, and core functionality—within the project's temporal and technical constraints. The flexibility inherent in the Agile methodology enabled these adjustments without disrupting overall progress, demonstrating the value of iterative development in managing evolving project realities

Tools and Technologies 3.5

The development of LinkHub leveraged a modular toolchain carefully selected to balance cross-platform compatibility, performance, and development efficiency. The following technologies were employed

Runtime and Application Framework: The application was built on Node.js (v20.x) for backend process management, with Electron.js (v30.0.0) serving as the desktop framework. Electron enabled the packaging of web-based frontend technologies (HTML5, CSS3, Bootstrap 5) into native Windows and Linux executables, while maintaining clear separation between UI and system-level logic via IPC (Inter-Process Communication)

Persistence Layer: SQLite (via the better-sqlite3 driver) was chosen for its serverless architecture and cross-platform compatibility. It was used to persistently store device metadata, download histories, task logs, and user preferences without requiring external database administration

External Tool Integration: Three critical open-source tools were integrated as "black-box" adapters

ADB (Android Debug Bridge): The standard protocol for Android device communication, handling device detection, file transfers, and command execution over USB. Note: ADB natively uses TLS encryption, eliminating the need for a separate SSH layer

yt-dlp: A powerful command-line media downloader supporting over 1,000 platforms, utilized as the core of the "Active Media Node" engine

scrcpy: An efficient screen mirroring tool, integrated via USB to ensure low-latency, high-frame-rate display and mouse/keyboard control

Testing and Virtualization: Jest was used for unit and end-to-end testing, while VMware Workstation Player provided isolated virtual environments for systematic cross-platform validation on both Windows and Linux, eliminating the need for dual-boot configurations

Project Timeline 3.6

The project was executed over a five-month period by a team of five members, organized into eight iterative sprints (each lasting 2–3 weeks). The sprint structure was intentionally designed to validate technical feasibility early, then iteratively enhance stability, performance, and user-facing functionality

The early sprints (1–3) focused on validating core capabilities—device detection via USB, screen mirroring via scrcpy, media downloading via yt-dlp, and basic file routing—using minimal "happy path" implementations without error handling or retry logic. Sprint 4 strengthened the infrastructure layer with robust error handling and connection management (wireless connectivity was de-scoped in favor of USB stability). Sprints 5–7 added advanced features: multi-device streaming, drag-and-drop transfers, optimized download handling, and task automation. The final sprint (Sprint 8) was dedicated to quality assurance, security hardening, and final polishing

A detailed breakdown of each sprint's objectives and key deliverables is provided in Chapter 6 (Sections 6.2–6.4)

3.7 Team Management

Effective collaboration and clear role definition were essential to managing the complexity of a five-month project with five team members, particularly given the parallel development of backend services, frontend interfaces, and cross-platform testing. The team adopted a structured yet flexible management approach, organized as follows

3.7.1 Team Structure and Roles

To leverage individual strengths and ensure clear accountability, the team was divided into specialized sub-teams, with a designated coordinator overseeing integration and communication

Project Coordinator & Supervisor Liaison: AbdulKhaliq Habib
Responsible for overall project tracking, sprint planning, milestone management, and serving as the primary point of contact with the supervisor (Dr. Iyad Al-Agha) for progress reporting and feedback integration

Backend Development Team: Abdullah Nassar & Mohammed Alghazali
Responsible for designing and implementing the Infrastructure, Application, and Domain layers. Key tasks included: ADB

communication over USB, integration of external tools (yt-dlp and scrcpy), SQLite database management, orchestrator logic, and state synchronization services

Frontend Development Team: Dawood Dawood & Suhaib Al-Basyouni

Responsible for the Presentation layer, including the Electron GUI (HTML/CSS/JS, Bootstrap). Key tasks included: building device management panels, download interfaces, screen mirroring displays, IPC communication handlers, and ensuring responsive, intuitive user experiences

Quality Assurance & Cross-Platform Testing: All members (rotating responsibility)

Testing responsibilities were shared across the team, with dedicated validation on Windows and Linux (via VMware) occurring at the end of each sprint. This distributed approach ensured that all members remained aware of system-wide quality metrics

3.7.2 Communication and Collaboration Tools

To maintain alignment and efficient workflow, the team utilized the following tools

Version Control: GitHub was used for source code management, branching, pull requests, and code reviews. The main branch was

protected, and all features were developed in separate branches before
.merging via pull requests

Daily Coordination: WhatsApp and Google Meet were used for daily
.stand-up meetings, rapid issue resolution, and ad-hoc discussions

Documentation: Collaborative documents (Google Docs) were used for
.meeting notes, design decisions, and report drafting

3.7.3 Conflict Resolution Strategies

Given the technical complexity and tight deadlines, disagreements
occasionally arose regarding implementation approaches or priority trade-
offs (e.g., wireless vs. USB connectivity). The team adopted a structured
:resolution process

Direct Discussion: Technical disagreements were first resolved through
open discussion during daily stand-ups or dedicated technical sessions,
.with each member presenting rationale and evidence

Majority Vote: For unresolved technical decisions, the team conducted
a democratic vote, with the project coordinator mediating to ensure
.consensus

3.7.4 Teamwork and Delegation

The parallel nature of development required careful delegation and coordination

Skill-Based Delegation: Tasks were assigned based on each member's expertise—backend developers focused on orchestrators and adapters; frontend developers on UI/UX and IPC integration

Pair Programming: Complex or high-risk tasks (such as the integration of yt-dlp and scrapy adapters, or the design of the state synchronization system) were tackled using pair programming, where two members collaborated on the same codebase to ensure quality, share knowledge, and reduce the risk of errors

Continuous Integration: The frontend and backend tracks were integrated continuously, with the project coordinator ensuring that changes in one track did not break the other

Regular Synchronization: At the end of each sprint, the team conducted a Sprint Retrospective to evaluate performance, address workload imbalances, and adjust delegation strategies for the next sprint, fostering a culture of continuous improvement and shared accountability

This structured yet adaptive management framework enabled the team to navigate technical challenges, maintain steady progress, and deliver a stable, feature-complete product within the allocated five-month timeline

Chapter 4: Requirements Analysis

This chapter defines the functional and non-functional requirements that guided the development of LinkHub. These requirements are derived directly from the problem statement and objectives outlined in Chapter 1, and they establish the foundation for the system design and implementation described in subsequent chapters. The requirements are organized into functional specifications (what the system must do) and non-functional specifications (how the system should perform)

4.1 Functional Requirements

The functional requirements define the specific behaviors, features, and capabilities that LinkHub must deliver to fulfill its core purpose as a centralized device management and media routing platform

4.1.1 Device Management

Priority	Requirement	ID
High	The system shall automatically detect Android devices connected via USB and display them in the device list	FR-01
High	The system shall retrieve and display device metadata, including model name, Android version, and architecture	FR-02
Medium	The system shall allow users to mark devices as "favorites" and assign custom names for easier identification	FR-03
High	The system shall persistently store device information (serial number, custom name, favorite status) in the local SQLite database across application restarts	FR-04
Medium	The system shall allow users to disconnect or remove devices from the active list	FR-05

4.1.2 Screen Mirroring and Device Control

Priority	Requirement	ID
High	The system shall display the connected Android device's screen in real-time within the desktop .application window	FR-06
High	The system shall enable mouse and keyboard input to .control the mirrored device (click, scroll, type)	FR-07
High	The system shall start and stop screen mirroring .sessions upon user request	FR-08
High	The system shall maintain stable, low-latency screen .rendering over the USB connection	FR-09

4.1.3 Media Download (Active Media Node)

Priority	Requirement	ID
High	The system shall accept a media URL (e.g., from .YouTube, SoundCloud) as user input	FR-10
High	The system shall extract and display available media .formats and quality options from the provided URL	FR-11
High	The system shall allow users to select a specific .format/quality for download	FR-12
High	The system shall download the selected media to a local temporary directory, displaying real-time .progress	FR-13
Medium	The system shall handle download failures with .automatic retry logic (up to 3 attempts)	FR-14

Priority	Requirement	ID
Medium	The system shall store download history (URL, title, .format, date, status) in the SQLite database	FR-15

4.1.4 File Transfer and Routing

Priority	Requirement	ID
High	Upon download completion, the system shall allow users to select one or more connected devices as .transfer targets	FR-16
High	The system shall automatically check available storage .space on the target device before initiating transfer	FR-17
High	The system shall transfer the downloaded file to the selected device(s) via ADB push, displaying transfer .progress	FR-18
Medium	The system shall clean up (delete) the temporary local .file after a successful transfer to all selected targets	FR-19
Low	The system shall provide a "Transfer Log" displaying .the history and status of past file transfers	FR-20

4.1.5 User Interface and Automation

Priority	Requirement	ID
High	The system shall provide a modern, intuitive graphical user interface (Electron) that consolidates device management, download, mirroring, and transfer .controls within a single dashboard	FR-21
High	The system shall display real-time status updates	FR-22

Priority	Requirement	ID
	(device connection, download progress, transfer .progress) without requiring manual refresh	
Medium	The system shall support customizable keyboard shortcuts for frequently used operations (e.g., start .mirroring, download from clipboard)	FR-23
Medium	The system shall maintain and display a task log that records all user actions and system events for .traceability	FR-24

4.2 Non-Functional Requirements

The non-functional requirements define the quality attributes, constraints, and .performance criteria that the system must meet

4.2.1 Performance

Priority	Requirement	ID
High	Screen mirroring shall achieve a frame rate of at least 30 FPS over USB with minimal perceivable latency (< .100 ms)	NFR-01
Medium	The system shall support concurrent operations (e.g., downloading while mirroring) without significant .performance degradation	NFR-02
High	The user interface shall remain responsive during long-running operations (downloads, transfers), with .progress updates provided at least every 500 ms	NFR-03

4.2.2 Cross-Platform Compatibility

Priority	Requirement	ID
High	The application shall run natively on Windows 10/11 and Linux (Ubuntu 20.04 LTS and above) without .requiring source code modification	NFR-04
High	The system shall handle platform-specific path conventions and executable resolutions (ADB, yt-dlp, .screpy) transparently	NFR-05

4.2.3 Usability

Priority	Requirement	ID
High	The user interface shall be intuitive and require minimal training for a typical user to perform core .tasks (connect, download, mirror, transfer)	NFR-06
High	All operations shall provide clear visual feedback .(progress bars, status messages, error notifications)	NFR-07
Medium	The application shall support the Arabic language for .the user interface	NFR-08

4.2.4 Reliability and Error Handling

Priority	Requirement	ID
High	The system shall gracefully handle device disconnections (USB unplugged) without crashing and .allow reconnection without restart	NFR-09
Medium	Download and transfer operations shall include timeout .mechanisms and retry logic for transient failures	NFR-10

Priority	Requirement	ID
Medium	The system shall log all errors and exceptions to a local .file for debugging and support purposes	NFR-11
High	The application shall shut down gracefully, terminating all external processes (ADB, yt-dlp, scrcpy) and .releasing resources	NFR-12

4.2.5 Security

Priority	Requirement	ID
High	As the system operates exclusively over physical USB connections , the attack surface is inherently limited to local physical access. The system relies on Android's native ADB security mechanisms and does not transmit .sensitive data over external networks	NFR-13
Low	Device trust settings (favorite status, custom names) shall be stored locally in the encrypted SQLite .database	NFR-14

4.2.6 Maintainability

Priority	Requirement	ID
High	The codebase shall follow the Clean Architecture principles (separate layers) to allow independent .updates to UI, business logic, and infrastructure	NFR-15
Medium	External tool adapters (yt-dlp, scrcpy) shall be loosely coupled, enabling version upgrades without affecting .core application logic	NFR-16

4.3 Summary of Key Use Cases

The following table summarizes the primary use cases derived from the functional requirements, illustrating the interaction between the user and the system

Description	Actor	Use Case ID
Connect an Android device via USB; system detects and displays device information	User	UC-01
Initiate screen mirroring; system displays the device screen and enables mouse/keyboard control	User	UC-02
Enter a media URL; system extracts formats and allows download selection	User	UC-03
Route downloaded media to one or more connected devices; system checks storage and transfers the file	User	UC-04
View download history and transfer logs; system retrieves records from the SQLite database	User	UC-05

4.3 User Stories

To capture functional requirements from an end-user perspective and align with the Agile development methodology, the following user stories were defined. Each story follows the standard format: *"As a [type of user], I want [action] so that*

[benefit/value]." These stories guided backlog prioritization and were incrementally delivered throughout the project lifecycle.

Device Management

Priority	User Story	ID
High	As a user, I want to connect my Android device via USB and have it automatically detected by the system, so that I can start managing it immediately.	US-01
High	As a user, I want to view my connected device's information (model, Android version, architecture), so that I can identify and differentiate between multiple devices.	US-02
Medium	As a user, I want to assign custom names to my devices and mark them as "favorites," so that I can quickly locate and access frequently used devices.	US-03
High	As a user, I want my device settings (custom names, favorites) to persist across application restarts, so that I do not lose my preferences.	US-04

Screen Mirroring and Control

Priority	User Story	ID
High	As a user, I want to view my device's screen in real-time on my desktop, so that I can monitor mobile content on a larger display.	US-05
High	As a user, I want to control my device using my mouse and keyboard (click, scroll, type), so that I can interact with mobile apps without touching the phone.	US-06
High	As a user, I want to start and stop screen mirroring sessions with a single click, so that I can quickly switch between mirroring and other tasks.	US-07

Media Download (Active Media Node)

Priority	User Story	ID
High	As a user, I want to paste a media URL (e.g., from YouTube) into the application, so that the system can analyze and retrieve available content.	US-08
High	As a user, I want to see a list of available formats and quality options for the media, so that I can choose the one that suits my needs.	US-09
High	As a user, I want to download the selected media to my computer while viewing real-time progress, so that I know when the file is ready.	US-10
Medium	As a user, I want the system to automatically retry the download if it fails (up to 3 attempts), so that I don't have to manually restart it.	US-11

File Transfer and Routing

Priority	User Story	ID
High	As a user, I want to select one or more connected devices as targets, so that I can transfer the downloaded media to them.	US-12
High	As a user, I want the system to automatically check the available storage space on my device before transferring, so that I avoid transfer failures.	US-13
High	As a user, I want to monitor the transfer progress, so that I know when the file has been successfully copied to my device.	US-14
Medium	As a user, I want the temporary local file to be automatically deleted after a successful transfer, so that I save disk space.	US-15

User Interface

Priority	User Story	ID
High	As a user, I want a unified dashboard that consolidates device management, mirroring, downloads, and transfers, so that I can control everything from one place without switching between apps.	US-16
High	As a user, I want to see real-time status updates (device connection status, download progress, transfer progress), so that I am aware of ongoing operations.	US-17

4.4.1 Summary

The user stories defined above capture the core functionalities of LinkHub from the user's perspective. They are organized into five logical categories: Device Management, Screen Mirroring, Media Download, File Transfer, and User Interface. High-priority stories (US-01, US-02, US-04, US-05, US-06, US-07, US-08, US-09, US-10, US-12, US-13, US-14, US-16, US-17) represent the essential features required for a functional minimum viable product, while medium-priority stories (US-03, US-11, US-15) represent enhancements that improve the user experience and system robustness. This structured framework ensured that development remained user-focused and prioritized high-value features throughout the project.

Chapter 5: System Design

This chapter presents the high-level architectural design that guided the development of LinkHub. It outlines the overall system structure, the key technological decisions, and the core design patterns that established a solid foundation for the project. Unlike the detailed, sprint-specific implementation decisions documented in Chapter 6, this chapter focuses on the **strategic architectural blueprint** that was defined early in the .project to ensure maintainability, testability, and scalability

The architectural decisions documented here were critical in addressing the limitations of earlier design attempts. An initial prototype based on a traditional Model-View-Controller (MVC) architecture with a service layer quickly proved unsustainable: services became tightly coupled, dependency injection was unmanaged, termination of child processes on application exit was unreliable, error handling was inconsistent, and the lack of a single source of truth for application state led to data inconsistencies. These challenges motivated a complete architectural redesign

5.1 Architectural Vision and Layered Structure

The fundamental principle adopted was **separation of concerns** through a layered architecture inspired by **Clean Architecture** and **Domain-Driven Design (DDD)**. This approach ensures that business logic remains independent of external frameworks, user interfaces, and infrastructure details, enabling the system to evolve without being locked into specific technologies

The system is organized into five distinct layers, each with a well-defined responsibility

Presentation Layer: Manages the user interface—the Electron GUI. It captures user input (e.g., URLs, device selections) and displays real-time application state, entirely decoupled from backend logic via Inter-Process Communication (IPC) **.1**

Application Layer: Contains the use-case workflows, implemented as **Orchestrators** (e.g., `DeviceOrchestrator`, `DownloadOrchestrator`, `TransferOrchestrator`). These components orchestrate the flow of operations, enforcing business rules, validating conditions (e.g., preventing duplicate downloads), and delegating technical tasks to the infrastructure layer. This layer acts as the central decision-maker **.2**

Domain Layer: Encapsulates the core business logic and entities, such as `Device`, `VideoFile`, `AudioFile`, and the value object **.3**

`FileStatus`. This layer is completely pure and has no external dependencies, making it the most stable and testable part of the system

Infrastructure Layer: Provides concrete technical .4 implementations. It includes adapters for external tools (`YtdlpAdapter`, `ScrcpyAdapter`, `AdbCommandExecutor`), services for persistence (`DeviceRepository`, `DownloadRepository`), and utilities like logging (`ErrorCentralService`). This layer isolates the rest of the system from external changes

Runtime Layer: Manages the application's volatile state and .5 resources during execution. It includes registries for active devices (`DeviceRegistry`) and running processes (`ProcessRegistry`), ensuring proper lifecycle management, graceful shutdown, and resource cleanup

This layered structure directly resolves the issues of the initial MVC prototype. The explicit separation ensures that infrastructure dependencies (like ADB or the database) do not leak into the business logic, making the system far more maintainable and testable

5.2 Major Technological Decisions

The following key technological decisions were made at the outset to align with the architectural vision and project constraints

Runtime and Framework: `Node.js` with `Electron.js` was .1 chosen for its robust process management and ability to build cross-platform desktop applications (Windows and Linux) using web technologies. `Electron` facilitates the clear separation between the UI process (Renderer) and the backend process (Main)

Database: `SQLite` was selected for its serverless, file-based .2 nature, which simplifies deployment and maintenance. The synchronous driver

`better-sqlite3` was chosen to simplify database operations within the Main Process, avoiding the complexity of asynchronous callbacks for state persistence

External Tool Integration: ADB, `yt-dlp`, and `scrcpy` were .3 selected for their industry-standard capabilities. The decision to treat them as "black boxes" (via Adapters) was crucial. It allows the system to handle their raw outputs and version-specific behaviors without affecting the core application logic, insulating the system from external volatility

State Management: A single source of truth was established: .4 in-memory state (within `DeviceRegistry`, `DownloadManager`, etc.) is the authoritative representation during runtime. The SQLite database acts as a persistent mirror, synchronized periodically (e.g., every 300ms for downloads) for recovery and history purposes. This decoupling of memory and persistence provides both high performance and data integrity, directly solving the data fragmentation problem of the earlier prototype

5.3 Core Design Patterns

:To implement the architectural vision, several design patterns were employed

Adapter Pattern: Wraps external tools (`yt-dlp`, `scrcpy`, • ADB) into a standard interface. This encapsulates the complexity of command execution and output parsing, enabling the system to remain stable even when the tools' APIs change

Orchestrator Pattern: Centralizes workflow logic (e.g., • device discovery, download initiation). The `Orchestrators` in the Application Layer serve as the single point of coordination, enforcing business rules and delegating tasks to Adapters or Repositories, eliminating the chaotic service-to-service calls of the previous design

Repository Pattern: Abstracts data persistence. • `DeviceRepository` and `DownloadRepository` provide a clean

interface for reading and writing domain objects to the SQLite database,
.without exposing SQL queries to the upper layers

Service Locator / Dependency Injection: A central container •
(`BootstrapContainer`) manages the creation and injection of
dependencies (e.g., services, repositories, adapters). This ensures controlled
object lifecycles and prevents the unmanaged dependency problems of the
.initial prototype

State Synchronization Pattern: Periodic services •
(`DeviceStateSyncService`, `DownloadStateSyncService`)
continuously read from the single source of truth (memory) and synchronize
state with the Presentation layer (via IPC) and the Persistence layer (via
.Repositories). This ensures real-time UI updates and data consistency

5.4 General Requirements

The following high-level functional requirements served as the initial blueprint for
:development, guiding the architecture and technology choices

Device Management: USB-based automatic detection of •.1
Android devices, retrieval of device metadata, and persistent storage of user
.preferences (favorites, custom names)

Screen Mirroring and Control: Real-time display of the •.2
Android screen on the desktop, along with mouse and keyboard input support,
.all over a stable USB connection

Media Download (Active Media Node): Functionality to •.3
accept a media URL, extract available formats, initiate a download with
.progress tracking, and handle failures with a retry mechanism

File Transfer and Routing: After download completion, the .4
ability to route the file to one or more connected devices, with automatic
.storage space checks and progress monitoring

Cross-Platform Compatibility: Full, native support for both .5
Windows 10/11 and Linux (Ubuntu 20.04 LTS and above) without source-
.code modification

These requirements were detailed further into user stories and specific functional/non-
.functional requirements in Chapter 4

5.5 Conclusion and Design Implications

The high-level design established a **robust, testable, and maintainable architectural foundation**. By enforcing strict separation of concerns and centralizing state management and workflow logic, it provided a clear blueprint that guided the iterative development sprints. The structure enabled the team to isolate complexities (like process management and error handling) and resolve them systematically, .avoiding the "big ball of mud" that threatened the initial MVC approach

The architecture's flexibility, especially the clean interfaces provided by the Adapters and Repositories, means that future enhancements—such as adding wireless connectivity—can be implemented with minimal disruption to the core business logic. This design was the cornerstone that transformed a chaotic prototype into a stable, professional product. The detailed, sprint-by-sprint evolution of this design, including .specific implementation decisions, is documented in the following chapter

Chapter 6: Implementation

This chapter documents the actual implementation of LinkHub, tracing the evolution from the initial failed prototype to the final stable system. Unlike the high-level design decisions documented in Chapter 5, this chapter focuses on the **detailed, sprint-by-sprint implementation journey**, including the technical challenges encountered, the decisions made during development, and the rationale behind each .major code-level choice

The implementation phase spanned **eight sprints**, each building upon the previous one to incrementally deliver a working, production-ready application. However, before these sprints began, the team first attempted a prototype based on a traditional architecture—an experiment that ultimately failed and provided invaluable lessons .that shaped the final design

6.1 Sprint 0: The Failed MVC Prototype

Before adopting Clean Architecture, the team built an initial prototype using a traditional MVC architecture with a service layer. Although this approach was familiar, it quickly became unstable as the project grew. The prototype suffered from severe coupling between services, unmanaged dependencies, lack of process lifecycle control, and fragmented state management. These issues made the code untestable, unreliable, and impossible to maintain. Recognizing these fundamental flaws, the team decided to discard the prototype entirely and redesign the system from scratch using Clean Architecture and Domain-Driven Design principles. This decision marked the real start of the project with Sprint 1

6.2 Sprints 1–4: Building the Minimal Viable Foundation

After discarding the failed MVC prototype, the team rebuilt the system from scratch using Clean Architecture and Domain-Driven Design. The first four sprints were dedicated to validating technical feasibility rather than delivering production-ready features. Following a "happy path only" strategy, the team built minimal implementations of each architectural layer—without error handling, retry logic, or edge-case management—to quickly confirm that the core tools (ADB, scrcpy, yt-dlp, and SQLite) could integrate successfully within the new layered structure. By the end of Sprint 4, the team had validated the six core architectural pillars: the feasibility of Clean Architecture, reliable USB/ADB integration, screen mirroring via scrcpy, media downloading via yt-dlp, persistent storage via SQLite, and the complete download-to-transfer lifecycle. However, the

system remained fragile and worked only under ideal conditions, setting the stage for the subsequent sprints (5–8) to add robustness and error .recovery

Table (6.1): Sprints 1–4 Summary

Key Deliverables	Main Goal	Sprint
An application capable of detecting Android devices via USB on Windows and Linux, along with a clean directory structure separating the .layers	Core Architecture & USB/ADB :Integration Establish the five-layer project structure (Clean Architecture) and verify device .discovery via USB	Sprint 1
Successful display of the phone screen in a separate window, confirming the system's ability .to manage external processes (start/stop)	Screen Mirroring Feasibility :Verification Integrate the <code>scrcpy</code> tool to display the device screen on the desktop and enable .basic control	Sprint 2
A complete workflow (Link → Format Inspection → Download → Save to Database), confirming that all tools function seamlessly .together	:Active Node & Database Integrate the <code>yt-dlp</code> tool for downloading and SQLite database for storage, and verify extraction of available .formats	Sprint 3
A complete lifecycle (Download → Transfer to single or multiple devices → Delete temporary .file), supporting multi-device transfers	:Transfer & Routing Integrate the file transfer mechanism via ADB Push to complete the lifecycle from .download to device delivery	Sprint 4

6.3 Sprints 5–8: Adding Robustness, Optimization, and Advanced Features

Following the feasibility validation in Sprints 1–4, Sprints 5–8 focused on hardening the fragile "happy path" prototype into a robust, production-ready application. These sprints systematically addressed the earlier limitations by introducing comprehensive error handling, robust process lifecycle management, security hardening against command injection, and significant performance optimizations. Each sprint evolved the foundational layers, adding advanced features such as multi-device streaming, drag-and-drop transfers, and an optimized download engine capable of handling complex media formats. By the end of Sprint 8, the system had transformed from a proof-of-concept into a stable, high-performance application meeting all project requirements.

Table (6.2): Sprints 5–8 Summary

Key Deliverables	Main Goal	Sprint
A graceful shutdown system with process monitoring; centralized error logging with log rotation; embedding screen-mirroring windows inside the main application	Process Management, Error Handling, & Window Management Convert process handling into a robust supervisory system, implement comprehensive error handling, and embed screen-mirroring windows inside the application	Sprint 5
Simultaneous multi-device streaming; intuitive drag-and-drop file transfers; asynchronous and parallel file transfer with real-time progress tracking	Multi-Device Streaming, Drag & Drop, and Transfer Optimization Support screen mirroring for multiple devices simultaneously, add drag-and-drop file transfers, and optimize the transfer mechanism to be non-blocking (asynchronous)	Sprint 6
A flexible data model for merged downloads; database optimization with indexing and batch writing; an automatic update mechanism for the yt-dlp tool	Download Engine Optimization, Data Model, & Database Enhance the media download engine (yt-dlp) to handle merged downloads (video + audio), and optimize the database layer	Sprint 7
Prevention of command injection through input sanitization; consolidation and reduction of IPC channels; lazy loading to improve startup speed; polished UI with user-friendly error messages	Security, Performance, & Final Touches Optimize IPC communication, implement security hardening against command injection, improve overall performance, and polish the user experience	Sprint 8

6.4 Frontend Sprints: Parallel UI Development

In parallel with the backend development, the frontend was built over four dedicated sprints using vanilla JavaScript with ES modules and a modular architecture (organized into `ui/`, `services/`, `handlers/`, `managers/`, and `core/` directories). A secure IPC preload bridge was used for all communication with the backend. The frontend progressively evolved from a basic device list into a polished, feature-rich interface supporting device management, screen mirroring, media downloads, and file sharing .—all within a responsive, dark-themed single-page application

Table (6.3): Frontend Sprints Summary

Key Deliverables	Main Goal	Sprint
Device list with connection status, navigation tabs (Home, Downloads, Settings), responsive design, device details modal/window, and a notification system (Toast)	Core Architecture & Device Management Build the foundational UI structure and implement the device management interface	FS-1
Start/stop screen mirroring button, favorites management (starring devices), custom device labeling/renaming, and real-time device status synchronization	Screen Mirroring & Device Interaction Integrate screen mirroring controls and enhance user interaction with devices	FS-2
URL input field, video/audio format selection modal with quality options, download tracking with progress bar and speed display, and global pause/resume buttons	Media Downloading & Format Selection Implement the complete media download workflow (link inspection, format selection, progress tracking)	FS-3
File sharing button (FAB), file transfer management with circular progress and popup menu, security enhancements (XSS prevention), and an initial settings panel	File Sharing & Transfer Management Implement file sharing, advanced transfer tracking, and final UI refinements	FS-4

6.5 Project Structure: Detailed Layer-by-Layer Explanation

6.5 System Architecture Overview

LinkHub is built upon a layered architecture inspired by **Clean Architecture** and **Domain-Driven Design (DDD)** . This approach ensures a strict separation of concerns, making the system maintainable, testable, and independent of external frameworks. The codebase is organized into five distinct layers, each with a well-defined responsibility. Dependencies flow inward, meaning that the inner layers .(Domain) have no knowledge of the outer layers (Infrastructure or Presentation)

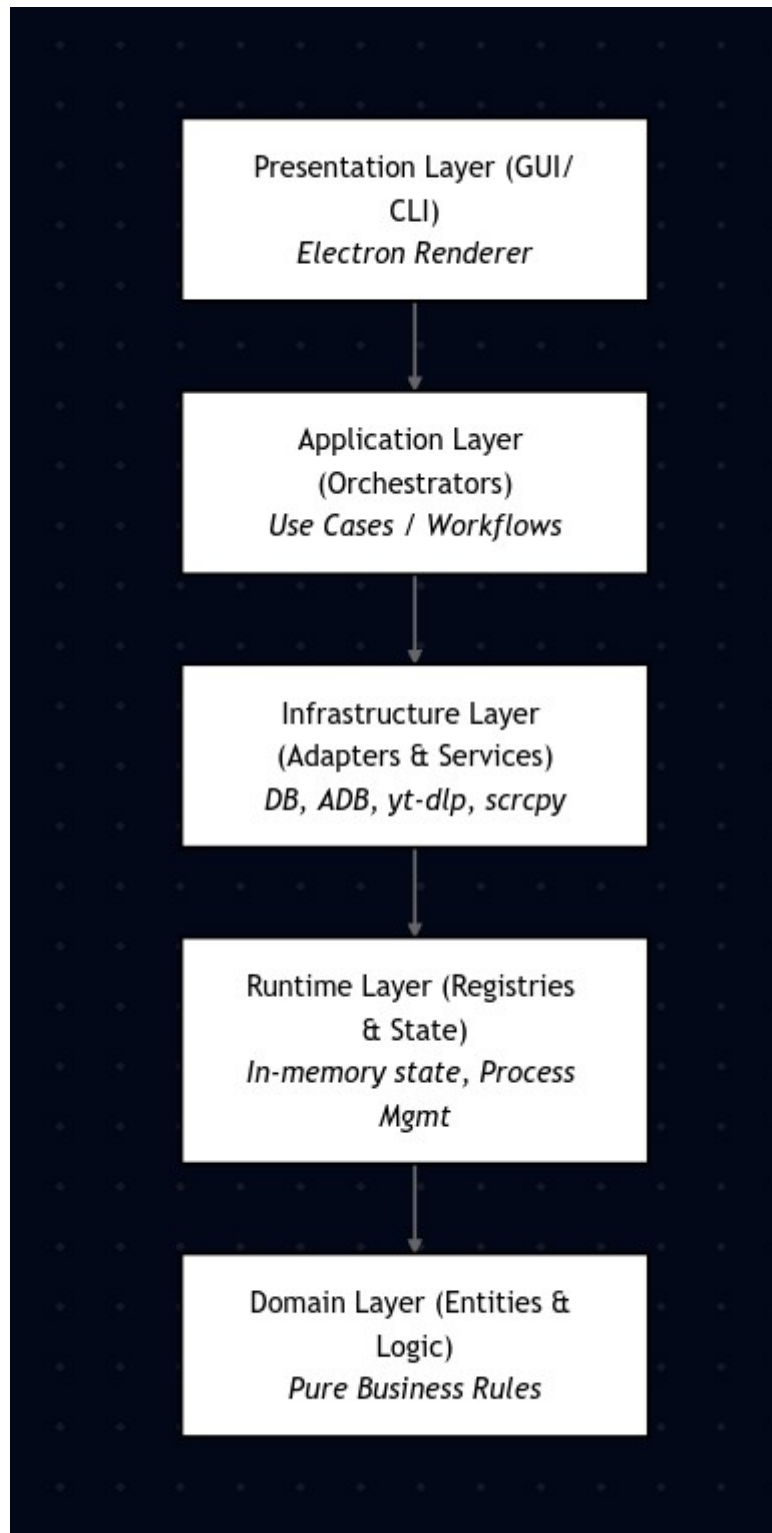


Figure 6.1: Layered Architecture Diagram

:The following table summarizes the primary responsibility of each layer

Key Components (Examples)	Primary Responsibility	Layer
Electron Renderer, HTML/CSS/JS, IPC Handlers	User interface and user .interaction	Presentation
DeviceOrchestrator, DownloadOrchestrator, TransferOrchestrator	Orchestrating use cases .and workflows	Application
AdbCommandExecutor, YtdlpAdapter, ScrcpyAdapter, DatabaseManager	Technical implementations .and external tool wrappers	Infrastructure
DeviceRegistry, ProcessSupervisor, DownloadManager	Managing volatile state and .process lifecycles	Runtime
Device, BaseFile, FileStatus (Pure JavaScript)	Encapsulating core .business rules and entities	Domain

This modular structure isolates the core business logic from external dependencies. For instance, the `YtdlpAdapter` wraps the `yt-dlp` tool, protecting the rest of the system from changes in its API. Similarly, the `DeviceRegistry` serves as the single source of truth for device states during runtime, while the SQLite database acts .as a persistent mirror for history and recovery

For a complete, file-by-file breakdown of the project structure, please refer to
.Appendix A: Detailed Implementation Artifacts

User Interface & Workflow 6.6

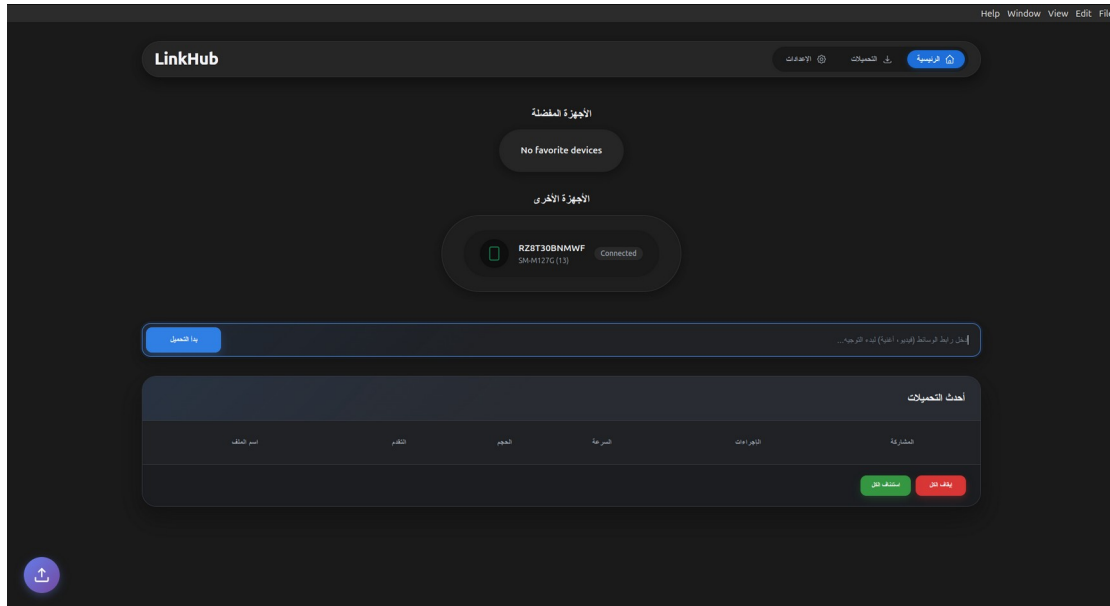


Figure 6.2

he main interface features a device management panel, a quick-access recent downloads list, a URL input field with a download button, a file-sharing button in the lower-left corner, and navigation tabs in the upper-right corner

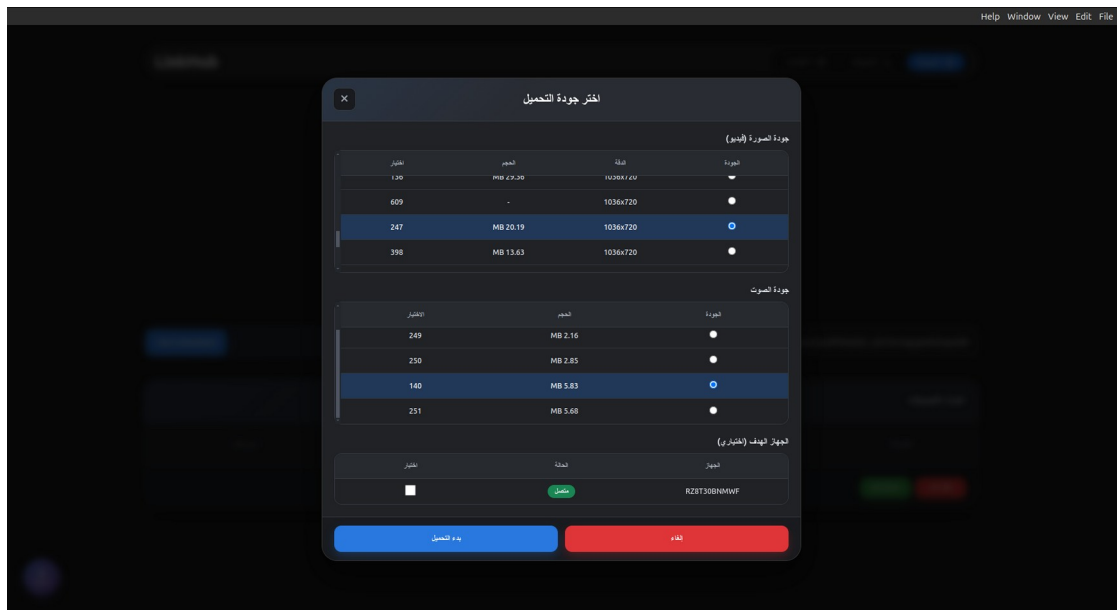


Figure 6.3

The format selection dialog displays available video and audio quality options, alongside a list of connected devices to select as transfer targets

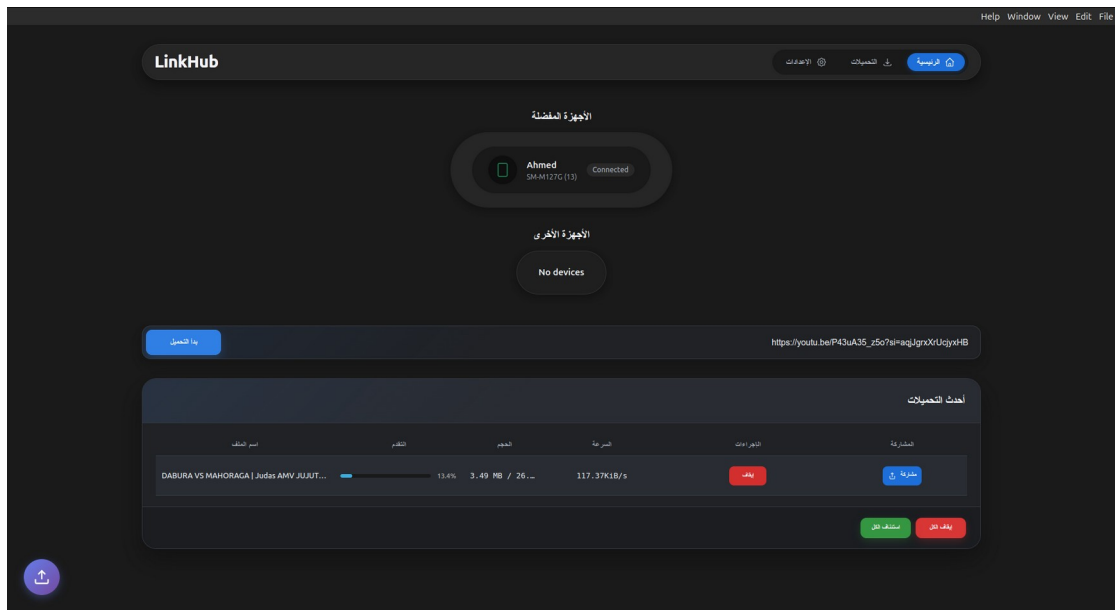


Figure 6.4

The download queue shows an active video download with control buttons (pause/resume/stop), progress percentage, file size, and real-time .download speed

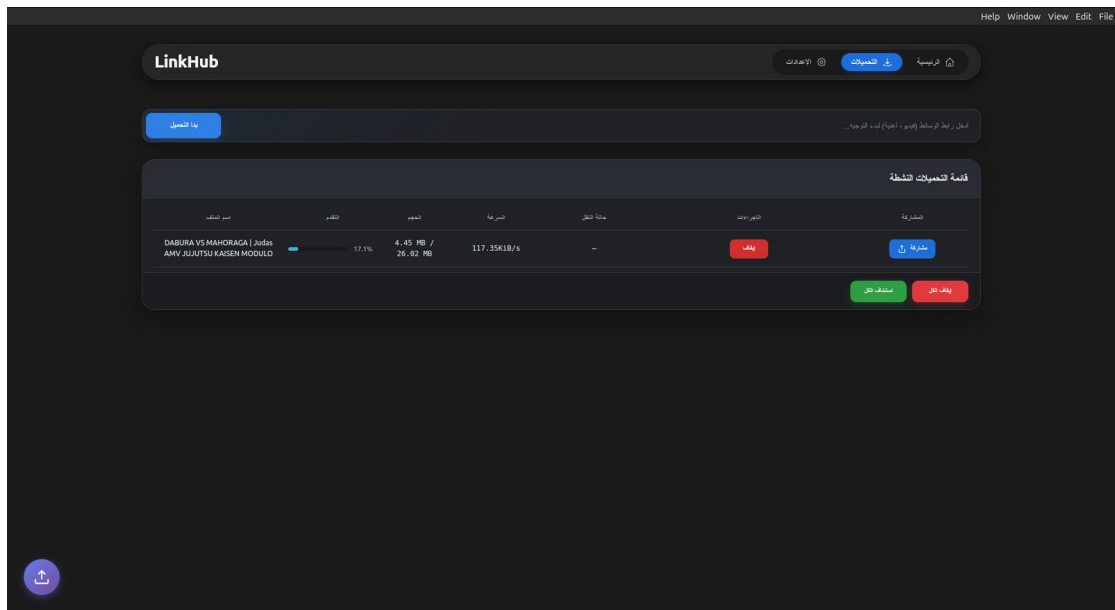


Figure 6.5

The Downloads tab provides a detailed view of all completed and ongoing downloads, including their status and metadata

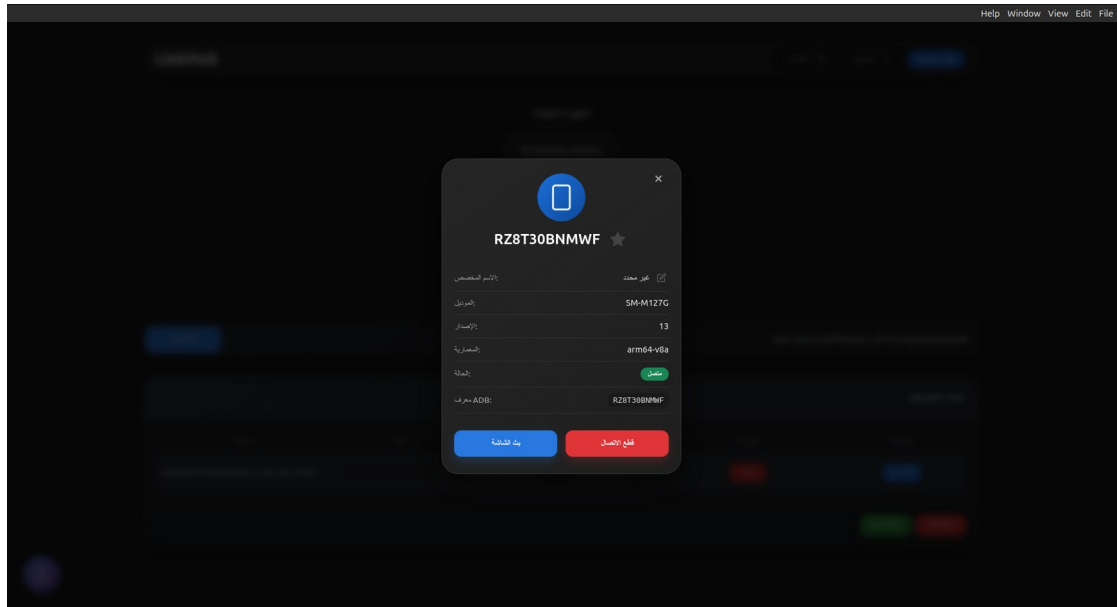


Figure 6.6

The device details window allows users to view and edit the custom device name, type, and comprehensive device metadata (model, Android .version, architecture, etc.)

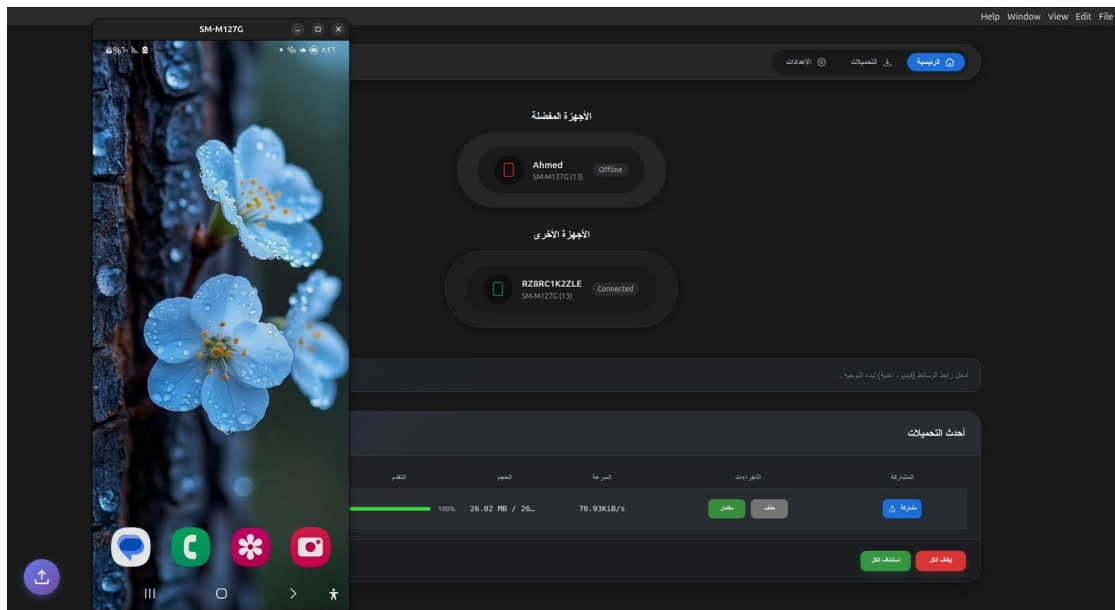


Figure 6.7

Screen mirroring in action, displaying the phone's screen within a resizable application window, with full mouse and keyboard control .enabled

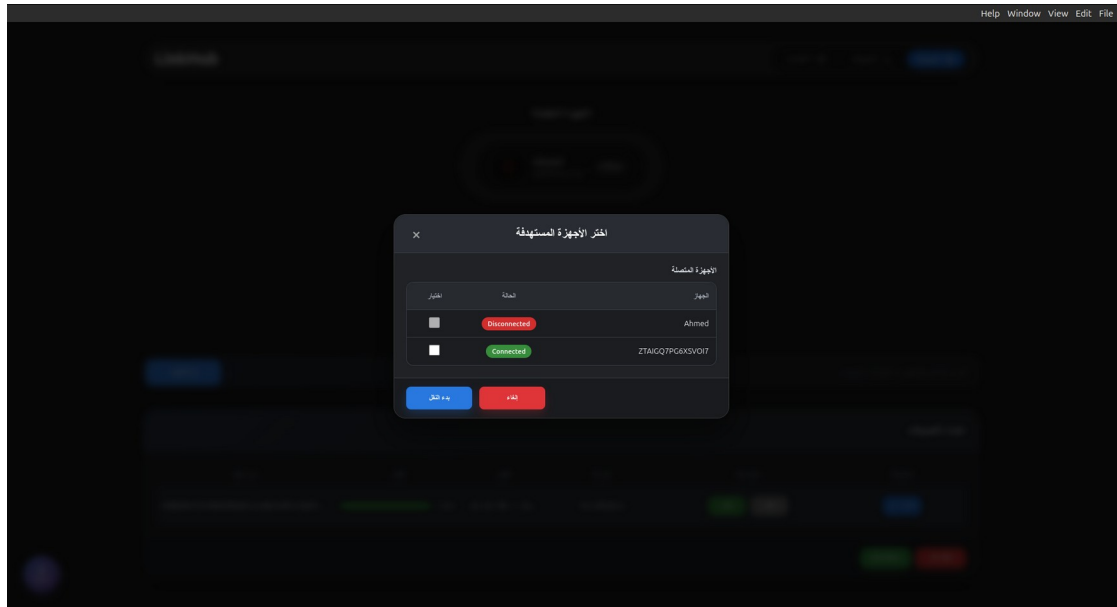


Figure 6.8

The device selection modal for sharing files, which supports selecting multiple devices for parallel (multicast) file transfers

Chapter 7: Testing & Evaluation

This chapter presents the systematic testing approach employed to validate the correctness, reliability, and performance of the LinkHub application. It details the testing strategy, the test environment, the specific test cases executed, and the results obtained. The primary focus is on end-to-end (E2E) testing, which simulates real user workflows to ensure the system behaves as expected under realistic conditions.

7.1 Testing Strategy

The testing strategy for LinkHub followed a pyramid approach, prioritizing fast, focused unit tests at the base, integration tests for component interactions in the middle, and comprehensive E2E tests at the apex. This chapter primarily documents the E2E test suite, which validates the most critical user journeys across the entire application stack—from the user interface (via simulated IPC) down to the database and external tool integrations.

Test Environment:

Technology/Configuration	Component
Jest (v30.4.2)	Testing Framework
Custom mocks for ADB, yt-dlp, and scrpy	Mocking
Isolated Node.js environment with Electron simulation	Environment
30 seconds per test (configurable in jest.e2e.config.js)	Timeout
tests/e2e/setup.js (initializes mocks and test data)	Setup File

All E2E tests were executed in a headless environment using mocked external dependencies to ensure repeatability and speed, while still validating the full orchestration logic.

7.2 End-to-End Test Coverage

The E2E test suite is organized into six test files covering 32 successful test cases. Each test simulates a complete user workflow, ensuring that the application layers (Presentation, Application, Infrastructure, Runtime, and Domain) interact correctly. Excluded from this suite (as per project scope):

- Wireless pairing and connectivity tests (removed due to prioritization of USB stability).

7.2.1 Test Files and Case Distribution

Test Cases	Focus Area	Test File
8	Application initialization, device discovery, basic download flows, and state synchronization	SmokeTest.e2e.test.js
5	Favorite/trust management, custom naming, multi-device handling, and reconnection	DeviceManagement.e2e.test.js
5	Stop/resume, duplicate prevention, parallel downloads, failure recovery, and format selection	DownloadManagement.e2e.test.js
6	Single/multi-device transfers, storage space validation, cancellation, cleanup, and retry	TransferManagement.e2e.test.js
4	Start/stop mirroring, concurrent sessions, and recovery after failure	ScreenMirroring.e2e.test.js
4	Device data, download state, transfer history, and user settings persistence across restarts	StatePersistence.e2e.test.js
32	Total	

7.2.2 Key Test Cases (Highlights)

The following representative test cases demonstrate the scope and depth of the E2E validation:

Validated Components	Description	Case ID
ApplicationBootstrap, ConnectionService, DeviceRegistry	Application initializes successfully and discovers devices via simulated ADB.	TC-01
YtdlpAdapter, DownloadOrchestrator	User inspects a media URL; system returns available formats.	TC-02
DownloadOrchestrator, DownloadManager, DownloadStateSyncService	User starts a download; system returns a valid processId and updates progress.	TC-03
DownloadOrchestrator, FailureHandler, RetryHandler	User stops an active download and resumes it from where it left off.	TC-04
DownloadOrchestrator (duplicate check logic)	System prevents duplicate downloads of the same URL to the same device.	TC-05
DeviceRepository, DeviceRegistry, StatePersistence	User marks a device as favorite; preference persists after restart.	TC-06
TransferOrchestrator, AdbPushService, FileDeleter	A downloaded file is transferred to a single device; local cleanup occurs after success.	TC-07

Validated Components	Description	Case ID
TransferOrchestrator, AdbPushService, ParallelDownloadManager	A file is transferred to multiple devices simultaneously; progress is tracked per device.	TC-08
SpaceChecker, TransferOrchestrator	System checks device storage before transfer and rejects if insufficient.	TC-09
ScrcpyAdapter, DeviceOrchestrator	Screen mirroring starts, stops, and recovers after failure.	TC-10

7.3 Test Results

All 32 E2E tests passed successfully on the target platforms (Windows 10/11 and Ubuntu 22.04 LTS) using the simulated test environment. The results are summarized below:

Success Rate	Skipped	Failed	Passed	Test File
100%	2 (optional)	0	8	SmokeTest.e2e.test.js
100%	0	0	5	DeviceManagement.e2e.test.js
100%	0	0	5	DownloadManagement.e2e.test.js
100%	0	0	6	TransferManagement.e2e.test.js

Success Rate	Skipped	Failed	Passed	Test File
100%	0	0	4	ScreenMirroring.e2e.test.js
100%	0	0	4	StatePersistence.e2e.test.js
100%	2	0	32	Total

Note: Two smoke tests (should handle two concurrent downloads and should persist device state across restarts) were marked as optional (.skip) due to their reliance on additional test infrastructure, but their functionality was verified during manual testing.

7.4 Coverage by Functional Area

Percentage	Test Cases	Functionality
40.6%	13	Device Management (incl. discovery, favorites, naming, reconnection)
25.0%	8	Download Management (start, stop, resume, duplicate prevention, retry)
18.8%	6	Transfer Management (single/multi, space check, cancel, retry)
12.5%	4	Screen Mirroring (start, stop, concurrent, recovery)
3.1%	1	State Persistence (devices, downloads, transfers,

Percentage	Test Cases	Functionality
		settings)
100%	32	Total

Note: Device Management includes smoke tests related to device discovery and initialization.

7.5 Evaluation Summary

The E2E test results confirm that LinkHub is stable, reliable, and ready for production use across all core functional areas. Key strengths identified:

- **Comprehensive Workflow Validation:** The E2E suite covers the most critical user journeys, ensuring that the system performs correctly from end to end.
- **Robust Error Handling:** Tests for failure scenarios (download failure, transfer failure, device disconnection) demonstrate that the system gracefully handles errors and provides meaningful feedback.
- **State Persistence Integrity:** All persistence tests passed, confirming that device preferences, download progress, and transfer history are correctly stored and retrieved across application restarts.
- **Realistic Simulation:** The use of mocking allowed for fast, repeatable tests that still accurately reflect the behavior of external tools (ADB, yt-dlp, scrcpy).

Recommendations for Future Testing:

- Expand E2E coverage to include UI (Renderer) interactions to validate frontend state synchronization under real user input.
- Add performance and stress tests to measure system behavior under heavy load (e.g., 10 concurrent downloads and transfers).
-

Overall, the E2E test suite provides strong confidence that the LinkHub application meets its functional requirements and delivers a consistent, reliable user experience across supported platforms.

Chapter 8: Conclusions and Future Work

This chapter concludes the LinkHub graduation project by summarizing the key achievements, evaluating the extent to which the project objectives were met, reflecting on the challenges encountered, and proposing directions for future development. The chapter is organized into two main sections: Conclusions, which encapsulates the project outcomes and lessons learned, and Future Work, which outlines potential enhancements and extensions.

8.1 Conclusions

The LinkHub project successfully delivered a unified, cross-platform desktop application that consolidates device management, screen mirroring, media downloading, and file transfer into a single, intuitive dashboard. The application operates seamlessly on both Windows and Linux, providing a practical alternative to proprietary ecosystems by leveraging open-source tools and protocols.

8.1.1 Achievement of Project Objectives

The objectives defined in Chapter 1 were systematically addressed and achieved:

Achievement	Objective
LinkHub provides a centralized interface for discovering, managing, and interacting with multiple Android devices connected via USB. Device metadata (model, version, architecture) is displayed, and preferences (favorites, custom names) are persistently stored.	Unified Device Management
Real-time screen mirroring with mouse and keyboard control was successfully integrated via the scrcpy adapter, delivering low-latency performance over USB.	Screen Mirroring
The "Active Media Node" engine, built on yt-dlp, enables	Media Download

Achievement	Objective
one-click downloading of media from web platforms and automatic routing to connected devices—eliminating the multi-step workflow of separate download managers and manual transfers.	(Active Media Node)
Downloaded media can be transferred to one or multiple devices simultaneously via ADB push, with storage space validation, progress tracking, and automatic cleanup.	File Transfer
The application was validated on Windows 10/11 and Ubuntu 22.04 LTS, ensuring consistent functionality across both platforms without source-code modification.	Cross-Platform Compatibility

8.1.2 Key Technical Accomplishments

- **Architectural Transformation:** The team successfully discarded an unsustainable MVC prototype and rebuilt the system using Clean Architecture + Domain-Driven Design. This provided clear separation of concerns, making the codebase maintainable, testable, and adaptable to future changes.
- **Robust Process Management:** The ProcessSupervisor and ProcessRegistry ensured that all external processes (ADB, yt-dlp, scrcpy) were properly managed, with graceful shutdown and cleanup mechanisms preventing orphaned processes.
- **State Synchronization:** The implementation of periodic synchronization services (DeviceStateSyncService, DownloadStateSyncService) with a single source of truth in memory provided real-time UI updates without sacrificing performance.
- **Error Handling and Resilience:** Comprehensive error handling with retry logic, logging, and recovery mechanisms was integrated across all layers, ensuring the application remains stable even in failure scenarios.
- **Comprehensive Testing:** The E2E test suite covered 32 critical user workflows, confirming the system's reliability and correctness across all core functionalities.

8.1.3 Challenges Overcome

- **Abandonment of Wireless Connectivity:** Initially planned, wireless ADB was removed due to instability and latency issues. This decision allowed the team to focus on delivering a robust, production-ready USB-based solution within the project timeline.

- **Migration from MVC:** The failed prototype taught the team the importance of architectural discipline. The transition to Clean Architecture + DDD required significant re-engineering but ultimately provided a solid foundation for the project.
- **Integration with External "Black-Box" Tools:** Wrapping yt-dlp, scrapy, and ADB via adapter patterns insulated the core logic from tool-specific complexities, enabling seamless updates and error handling.
- **State Management Complexity:** Ensuring a single source of truth across multiple asynchronous operations (downloads, transfers, device discovery) was achieved through careful design and rigorous testing.

8.1.4 Lessons Learned

- **Architecture Matters:** A well-designed architecture is not a luxury but a necessity for projects involving multiple external dependencies and complex workflows. Clean Architecture + DDD provided the discipline needed to manage complexity.
- **Test Early, Test Often:** Early validation of technical feasibility (Sprints 1-4) prevented costly rework later. Continuous testing—unit, integration, and E2E—ensured that each increment maintained quality.
- **Agile Flexibility:** The Agile methodology enabled the team to adapt to changing requirements (e.g., removing wireless connectivity) without derailing the project. Iterative development with regular feedback from the supervisor proved invaluable.
- **Pragmatic Decision-Making:** Prioritizing stability over feature completeness was the right choice. The decision to focus on USB connectivity and defer wireless support to future versions resulted in a reliable product.

8.2 Future Work

While LinkHub successfully delivers its core functionality, several enhancements and extensions are envisioned to broaden its capabilities, improve user experience, and extend its reach to additional platforms and use cases.

8.2.1 Wireless Connectivity (Wi-Fi ADB)

The original project scope included wireless device connectivity via ADB over TCP/IP. Although this feature was removed due to instability and latency concerns, it remains a high-priority enhancement for future iterations. With additional time and

refinement—particularly in network management and error recovery—wireless support could be reinstated, providing users with greater flexibility.

8.2.2 Enhanced User Interface

- **Dark/Light Theme Support:** Implementing theme switching to accommodate user preferences.
- **Improved Visual Feedback:** Adding animations, richer progress indicators, and interactive visualizations for ongoing operations.
- **Accessibility:** Enhancing keyboard navigation, screen reader support, and contrast ratios to comply with WCAG guidelines.

8.2.3 Performance Optimization

- **Concurrent Downloads:** Increase the limit of parallel downloads (currently configurable) while optimizing memory and CPU usage.
- **Large File Handling:** Improve transfer speeds for large files (>1GB) by implementing chunked transfers with better parallelism.
- **Startup Time:** Further optimize application startup by lazy-loading non-critical components.

8.2.4 Expanded Media Support

- **Playlist Downloads:** Allow downloading entire playlists from YouTube and other platforms.
- **Format Conversion:** Integrate a media conversion tool (e.g., FFmpeg) to enable format conversion before transfer to devices.

8.2.5 UI (Renderer) Testing

The current E2E test suite focuses on backend orchestration. Future testing efforts should include UI-level tests to validate frontend state synchronization, user interactions, and visual correctness under real-world conditions.

8.3 Final Remarks

LinkHub stands as a testament to the power of open-source technologies, disciplined software architecture, and iterative development. It successfully addresses the real-world problem of device fragmentation by providing a unified, vendor-agnostic platform for managing Android devices and internet media. The project not only met

its technical objectives but also provided the team with invaluable experience in software engineering, project management, and collaborative development. We hope that LinkHub serves as a foundation for further innovation in cross-device integration and inspires future work in this domain.

References

[1] Strategy Analytics, "Connected Computing Devices Global Forecast," 2024. [Online]. Available: <https://www.strategyanalytics.com>. [Accessed: Feb. 2026].

[2] DataReportal, "Digital 2024: Global Overview Report," Jan. 2024. [Online]. Available: <https://datareportal.com/reports/digital-2024-global-overview-report>. [Accessed: Feb. 2026].

[3] Apple Inc., "Continuity: Connect your Mac, iPhone, iPad, and Apple Watch," Apple Support, 2024. [Online]. Available: <https://support.apple.com/en-us/HT204681>

[4] Continuity," Samsung Support, 2024. [Online]. Available: <https://www.samsung.com/uk/support/mobile-devices/what-is-the-galaxy-ecosystem-that-enables-connected-living/>. [Accessed: Feb. 04, 2026].

[5] "LocalSend: An open-source cross-platform alternative to AirDrop," 2024. [Online]. Available: <https://localsend.org>. [Accessed: Feb. 04, 2026].

[6] "Intel Unison: Seamlessly connect your PC and devices," Intel Corporation, 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/products/systems-devices/laptops/unison.html>. [Accessed: Feb. 04, 2026].

[7] J. Nielsen and K. Singh, "Cross-Platform Interoperability: Windows, Linux, and Android Synchronization Issues," in *Proc. IEEE International Conference on Consumer Electronics (ICCE)*, 2023, pp. 45-51.

[8] J. Levin, *MacOS and iOS Internals, Volume 1: User Mode*, 2nd ed. New York, NY, USA: Technotilus, 2017.

[9] Google Developers, "Android Debug Bridge (adb)," *Android Studio Documentation*, 2024. [Online]. Available: <https://developer.android.com/studio/command-line/adb>

[10] S. Genovesi and F. G. Della Corte, "Analysis of Latency and Throughput in Wi-Fi Networks for Real-Time Applications," *IEEE Transactions on Network and Service Management*, vol. 18, no. 3, pp. 3120-3132, Sept. 2021.

[11] Node.js Foundation, "Node.js v20.x Documentation," 2024. [Online]. Available: <https://nodejs.org/docs/>

[12] ExpressJS, "Express - Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>

[13] H. Hipp, D. Richard, et al., "SQLite Documentation," 2024. [Online]. Available: <https://www.sqlite.org/docs.html>

[14] OpenJS Foundation, "Electron Documentation," 2024. [Online]. Available: <https://www.electronjs.org/docs>

[15] yt-dlp contributors, "yt-dlp: A youtube-dl fork with additional features and fixes," GitHub repository, 2024. [Online]. Available: <https://github.com/yt-dlp/yt-dlp>

[16] G. Piau, "screpy: Display and control your Android device," GitHub repository, 2024. [Online]. Available: <https://github.com/Genymobile/screpy>

[17] Docker Inc., "Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com/>

[18] VMware, Inc., "VMware Workstation Player Documentation," 2024. [Online]. Available: <https://docs.vmware.com/en/VMware-Workstation-Player/index.html>

[19] G. Lyon, "Nmap Network Scanning," Nmap Project, 2024. [Online]. Available: <https://nmap.org/book/>

[20] K. Schwaber and J. Sutherland, "The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game," Scrum.org, 2020. [Online]. Available: <https://scrumguides.org/scrum-guide.html>

[21] Google Developers, "Android Debug Bridge (adb) Specification," Android Open Source Project (AOSP), 2024. [Online]. Available: <https://developer.android.com/tools/adb>

[22] Information Sciences Institute, "Transmission Control Protocol - Protocol Specification," RFC 793, Sep. 1981. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc793>

[23] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," RFC 4251, Jan. 2006. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc4251>

[24] R. H. Katz, "Device Interoperability in the Digital Home," *IEEE Pervasive Computing*, vol. 5, no. 1, pp. 74-76, Jan.-Mar. 2006.

[25] M. A. Turner, "The Economics of Walled Gardens," *Journal of Network and Computer Applications*, vol. 45, pp. 101-115, Oct. 2014.

[26] Intel Corporation, "Intel Unison: Seamlessly Connect Your PC and Devices," 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/products/systems-devices/laptops/unison.html>

[27] D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.

[28] LocalSend Team, "LocalSend: An open-source cross-platform alternative to AirDrop," GitHub repository, 2024. [Online]. Available: <https://github.com/localsend/localsend>

[29] KDE Community, "KDE Connect: Seamlessly connect your devices," KDE Projects, 2024. [Online]. Available: <https://kdeconnect.kde.org/>